



多线程

▼ 多线程使用

从 C++11 标准开始，C++ 官方提供了 `<thread>` 头文件，彻底正规化了多线程写法。

在 C++ 中，每个线程接受一个函数，所以线程的执行基本是以函数为载体。

多线程一般有以下优势：

- **CPU 利用率更高**：多核 CPU 可以真正并行处理多个任务。
- **程序响应更快**：UI 程序不卡死；服务器可以同时处理多个请求。
- **结构更清晰**：逻辑上独立的任务可以用独立线程管理。

但是，多线程存在同时访问同一资源的情况，此时就会产生**线程安全**、**资源竞争**、**死锁**等问题。

▼ 创建线程

线程的创建与使用主要依赖于以下三种方法：

- `std::thread`：数据类型，创建一个线程对象，对象在创建时自动启动线程。
- `join()`：主线程阻塞等待子线程结束。
- `detach()`：子线程与主线程分离，自己运行，主线程不等待它，子线程执行完自己退出（但小心野线程问题）。

```
#include <iostream>
#include <thread>

void task() {
    std::cout << "Task is running" << std::endl;
}

int main() {
```

C++

```
std::thread t(task); // 新建线程执行task()
t.join();           // 主线程等待t结束
return 0;
```

线程接受的函数，可以是定义好的函数，也可以是匿名函数（Lambda 表达式）。

常见野线程：

场景	解释
主程序结束了，但子线程还在跑	主线程退出，子线程被操作系统强制杀死或放到后台
局部变量作为线程入口参数，但函数返回后，变量销毁了	线程访问了已经释放的内存
线程对象 <code>std::thread</code> 在还没 join 就析构了	<code>C++</code> 标准规定这种情况程序直接终止

▼ 线程传参

- 传值：

```
void print(int x) {
    std::cout << "Value: " << x << std::endl;
}

int main() {
    std::thread t(print, 42);
    t.join();
}
```

C++

- 传引用：要用 `std::ref` 包装引用参数！否则线程会拷贝副本，不是修改原变量！

```
#include <functional>

void update(int& x) {
    x += 10;
}

int main() {
    int num = 5;
    std::thread t(update, std::ref(num));
    t.join();
    std::cout << "Updated num: " << num << std::endl;
}
```

C++

▼ 线程同步

▼ 数据竞争

多个线程如果同时访问/修改同一资源，就会出**数据竞争 (Data Race)**，程序结果不可预测。

比如：

```
C++

int counter = 0;

void increment() {
    for (int i = 0; i < 10000; ++i) {
        counter++; // 多线程一起改，数据竞争
    }
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "counter = " << counter << std::endl;
}
```

`counter++` 不是原子操作，它包含：

1. 读 `counter`
2. 加 1
3. 写回 `counter`

如果两个线程完全在某一时刻完全同步，同时读取 `counter`，那么写回的 `counter` 只增加了 1，但却算作两次循环，所以最终 `counter` 的值将会低于 `10000`。

可以使用 `<atomic>` 库中的 `std::atomic<T>` 将数据定义为原子类型，它提供了多线程间的原子操作。

原子类型是封装了一个值的类型，它的访问保证不会导致数据的竞争，并且可以用于在不同的线程之间同步内存访问。

如果不使用原子类型，就需要使用互斥锁避免资源竞争。

▼ 互斥锁

在多线程中，为了防止数据竞争，会使用互斥锁。

当前持有独占锁的进程会优先进行，其他进程需要等待当前进程结束。

前提是进程函数使用同一个锁：

- 持有不同锁实例的进程不会互相影响：

```
▼ 持有不同锁实例 C++

#include <iostream>
#include <mutex>
#include <thread>
```

```

std::mutex mtx1, mtx2; // 两个互斥锁
int shared_data1 = 0;
int shared_data2 = 0;

void func_with_lock1() {
    std::unique_lock<std::mutex> lock(mtx1); // 获取 mtx1 锁
    shared_data1 = 42;
    std::cout << "func_with_lock1 is modifying shared_data1\n";
}

void func_with_lock2() {
    std::unique_lock<std::mutex> lock(mtx2); // 获取 mtx2 锁
    shared_data2 = 100;
    std::cout << "func_with_lock2 is modifying shared_data2\n";
}

int main() {
    std::thread t1(func_with_lock1);
    std::thread t2(func_with_lock2);

    t1.join();
    t2.join();

    return 0;
}

```

`mtx1`，`mtx2` 是两个锁实例，只有持有相同锁的进程才会互相影响。

- 未持锁的进程不受锁的影响：

▼ 未持锁 C++

```

#include <iostream>
#include <mutex>
#include <thread>

std::mutex mtx;
int shared_data = 0;

void thread_with_lock() {
    std::unique_lock<std::mutex> lock(mtx);
    shared_data = 100;
    std::cout << "thread_with_lock is modifying shared_data\n";
}

void thread_without_lock() {
    // 没有锁的线程直接访问共享资源，可能会发生数据竞争
    std::cout << "thread_without_lock is reading shared_data: " << shared_data << std::endl;
}

int main() {
    std::thread t1(thread_with_lock);
    std::thread t2(thread_without_lock);
}

```

```

std::thread t3(thread_without_lock);

t1.join();
t2.join();
t3.join();

return 0;
}

```

未使用锁的函数不受独占锁的影响，会直接访问数据。

持有锁是阻塞其他线程获取同一锁的根本原因。

是不是访问同一份数据，程序不知道也不关心，程序只管谁持有了锁。

锁的阻塞作用，跟访问了哪份数据无关，只跟加的是不是同一把锁有关。

也就是说，如果为并未访问同一数据的多线程使用同一个锁，也会触发线程锁，导致效率降低，所以在程序设计时需要考虑线程到底会不会出现资源竞争。

为了最大化效率，可以只在访问所有线程都可能访问的资源时再加锁。

需要注意的时，标准输入和标准输出也是公共资源，多线程中如果使用建议加锁。

▼ std::mutex

std::mutex 是最基本的锁，同一时刻只有一个线程持有锁。如果一个线程加锁了，其他线程必须等待。

std::mutex 可以确保同一时刻只有一个线程访问共享数据，从而避免数据竞争。

```

#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx; // 创建一个互斥锁
int counter = 0;

void increment() {
    for (int i = 0; i < 10000; ++i) {
        mtx.lock(); // 加锁
        counter++;
        mtx.unlock(); // 解锁
    }
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "counter = " << counter << std::endl;
}

```

- **mtx.lock()**：获取锁，如果已经被别的线程锁了，会等待。

C++

- `mtx.unlock()`：释放锁，让其他线程可以继续。

需要注意，使用 `std::mutex` 时，有可能在函数异常退出时漏掉 `unlock`。

- `std::mutex` 内部通常依赖操作系统提供的原子同步机制（比如 Linux 下是 pthread mutex，Windows 下是 Critical Section 或 Kernel Object）。
- 当一个线程调用 `lock()` 时，如果锁是空闲的，它就立即获取锁；
- 如果锁已经被别的线程占用，那么它会阻塞挂起（sleep）等待锁释放；
- `unlock()` 释放锁后，操作系统会唤醒等待中的一个线程。

由于 `mutex` 是重量级锁，开销不小（尤其在竞争激烈时），所以不建议锁住太大的代码块（锁粒度尽量小）。

▼ `std::recursive_mutex`

支持递归加锁，同一线程可以多次 `lock`，内部会有一个计数器。`unlock` 时每调用一次，计数器减 1，直到计数为 0 才真正释放锁。

```
▼ std::recursive_mutex C++
std::recursive_mutex rmtx;

void recursive_function(int depth) {
    if (depth <= 0) return;
    rmtx.lock();
    recursive_function(depth - 1);
    rmtx.unlock();
}
```

适合：

- 递归函数加锁
- 复杂对象内部函数互相调用时需要加锁

注意：

- 多次 lock 要对应多次 unlock！
- 不要滥用，容易隐藏逻辑错误。

▼ `std::timed_mutex`

带超时的普通锁，支持尝试加锁一段时间，如果失败就返回，不会一直挂死。

```
▼ std::timed_mutex C++
std::timed_mutex tmtx;
if (tmtx.try_lock_for(std::chrono::milliseconds(100))) {
    // 成功加锁
    // critical section
    tmtx.unlock();
} else {
    // 100ms内没拿到锁，做别的事情
}
```

适合：

- 避免长时间等待，防止死锁卡住
- 超时可以自己采取措施（如重试、跳过、报错等）

▼ std::recursive_timed_mutex

`recursive_mutex` 和 `timed_mutex` 的结合体：

- 支持递归多次加锁
- 每次加锁都可以带超时限制

```
▼ std::recursive_timed_mutex C++
std::recursive_timed_mutex rtm;
if (rtm.try_lock_for(std::chrono::milliseconds(50))) {
    rtm.lock(); // 再锁一次 (递归)
    // ...
    rtm.unlock();
    rtm.unlock();
}
```

▼ std::shared_mutex

多读单写锁，允许多个读者（shared lock）同时持有锁，写者（exclusive lock）独占锁，必须等所有读者释放后才能写。

```
▼ std::shared_mutex C++
#include <shared_mutex>

std::shared_mutex smtx;

void reader() {
    std::shared_lock<std::shared_mutex> lock(smtx); // 共享读锁
    // safe read
}

void writer() {
    std::unique_lock<std::shared_mutex> lock(smtx); // 独占写锁
    // safe write
}
```

适合读多写少的场景（比如缓存、查询系统）。

▼ std::lock_guard

为了避免 `mutex` 未 `unlock` 的问题（比如函数异常退出时漏掉 `unlock`），可以用 RAII 机制自动管理锁。

`std::lock_guard`，以及 `std::unique_lock` 与 `std::shared_lock` 并不是锁本身，而只是锁的管理器。

每个进程都可以有自己的管理器，但是只有一个管理器真正持有锁，其他想要持有同一个锁的管理器会阻塞进程。

管理器在被创建时就会尝试去获取锁，若获得则继续执行，否则阻塞在管理器初始化处。

管理器在生命周期结束时会自动释放其获取的锁，所以不需要手动释放锁。

```
C++

#include <mutex>

void increment() {
    for (int i = 0; i < 10000; ++i) {
        std::lock_guard<std::mutex> lock(mtx); // 构造时自动加锁，析构时自动解锁
        counter++;
    }
}
```

一旦 `lock` 对象作用域结束（比如函数返回或异常），锁自动释放，不容易出错。

▼ `std::unique_lock`

比 `lock_guard` 更灵活，比如可以延迟加锁、提前释放锁，并且同样保持了 RAII 机制自动管理。

在使用时建议创建一个新的作用域，在该作用域结束时会自动解锁。

```
C++

while (true) {
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        // 使用 while 确保在虚假唤醒时线程不会继续执行
        while (task_queue.empty() && !done) {
            cv.wait(lock); // 阻塞直到有任务或完成标志
        }

        if (task_queue.empty() && done) {
            break; // 如果任务队列为空并且 done 为 true，退出循环
        }

        path = task_queue.front();
        task_queue.pop();
    }
    // others
}
```

即使已经在 `while` 循环的作用域中，但是在使用锁时依然创建了一个新的作用域。

特性	<code>std::lock_guard</code>	<code>std::unique_lock</code>
立即加锁	是	可以选择是否立即加锁 (<code>defer_lock</code>)
延迟加锁	否 (构造就加锁)	支持延迟加锁
提前解锁 / 重新加锁	否	支持 <code>.unlock()</code> / <code>.lock()</code>
移动赋值 (转移锁的控制权)	否	支持 (可把锁对象转移)
与条件变量配合 (灵活释放锁)	有限制	非常适合

- 延迟加锁 (`defer_lock`): `lock_guard` 一创建就必须加锁; 而 `unique_lock` 可以选择暂时不加锁, 后面再手动加锁。

```

▼ 延迟加锁 C++
std::mutex mtx;

// 立即加锁
std::lock_guard<std::mutex> lock(mtx);

// 延迟加锁 (defer_lock)
std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
// ... 做一些事情
lock.lock(); // 手动加锁

```

- 可以提前 unlock: `lock_guard` 不能提前解锁 (除非人为析构), `unique_lock` 支持在中途随时解锁。

```

▼ 提前 unlock C++
std::unique_lock<std::mutex> lock(mtx);
do_something(); // 持锁
lock.unlock(); // 手动提前释放锁
do_other_thing(); // 不持锁

```

- 可以 lock()/unlock() 多次: `lock_guard` 只允许构造时加锁, 之后不能再重新加锁; `unique_lock` 可以动态 lock()/unlock() 多次。

```

▼ lock()/unlock() 多次 C++
std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
lock.lock(); // 第一次加锁
lock.unlock(); // 解锁
lock.lock(); // 重新加锁

```

- 可以移动 (move) 锁对象: 因为 `unique_lock` 可以移动构造和移动赋值, 所以它可以在函数之间传递锁的所有权。

```

▼ 移动 (move) 锁对象 C++
std::unique_lock<std::mutex> create_lock() {
    std::unique_lock<std::mutex> lock(mtx);
    return lock; // move出去
}

```

```
void use_lock(std::unique_lock<std::mutex> lock) {
    // 使用传进来的锁
}
```

- 配合条件变量 (condition_variable) 使用：

在使用 `std::condition_variable` 时，通常要求：

- 先解锁 (释放 mutex)
- 等待通知
- 被唤醒后重新加锁

而 `std::unique_lock` 是唯一能很好配合 `std::condition_variable::wait()` 的锁。

▼ 配合条件变量

C++

```
std::condition_variable cv;
std::mutex mtx;
bool ready = false;

void worker() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return ready; }); // 自动解锁等待，唤醒后自动加锁
    std::cout << "Worker runs\n";
}
```

`lock_guard` 不能用于 `wait()`，因为 `wait()` 需要在内部暂时释放锁。

▼ std::shared_lock

`std::shared_lock` 只能持有 `std::shared_mutex`，并且进程函数不能对数据进行写入操作。

- 多个线程可以同时持有锁（只读）。
- 共享锁之间不会互相阻塞，但不能与独占锁同时存在。
- 用于读取数据的临界区。

▼ 条件变量 std::condition_variable

`std::condition_variable` 是 C++11 引入的线程同步原语，主要用于让一个或多个线程等待某个条件发生，一旦条件满足，就唤醒线程继续执行。

需要和 `std::unique_lock` 与 `std::shared_lock` 配合使用，不能直接接受锁本身作为参数，而是接受一个锁的管理器。

`wait` 函数接受一个锁的管理器实例，以及一个返回值为 `bool` 的函数。调用 `wait` 后，会进行判断，若函数返回为 `true`，则继续执行，否则阻塞进程，中断在 `wait` 调用处并让出锁。

其他线程调用 `notify_one()` 或 `notify_all()` 时，阻塞在 `wait` 的线程会再次进行判断，若为真，则取得锁(或等待锁被让出后取得锁)并从 `wait` 函数(之前中断处)继续执行。

▼ 示例

C++

```
#include <iostream>
#include <thread>
#include <mutex>
```

```

#include <condition_variable>

std::mutex mtx;
std::condition_variable cv;
bool ready = false; // 共享条件

void worker() {
    std::unique_lock<std::mutex> lock(mtx);
    // 等待 ready == true
    cv.wait(lock, [] { return ready; });

    // 被唤醒且 ready==true, 继续工作
    std::cout << "Worker thread is processing data..." << std::endl;
}

void setter() {
    std::this_thread::sleep_for(std::chrono::seconds(1)); // 模拟做一些准备工作
    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true; // 设置条件为 true
    }
    cv.notify_one(); // 通知等待的线程
}

int main() {
    std::thread t1(worker);
    std::thread t2(setter);

    t1.join();
    t2.join();

    return 0;
}

```

关键点解释：

- `std::unique_lock<std::mutex> lock(mtx);`
 - 必须加锁，`wait` 函数内部要操作 mutex。
- `cv.wait(lock, predicate);`
 - `predicate`（谓词）是一个返回 bool 的函数。只有 `predicate==true` 时才继续，否则挂起等待。
 - 这个谓词是为了防止**虚假唤醒**（spurious wakeup）。
- `cv.notify_one();`
 - 唤醒一个等待中的线程。
- `cv.notify_all();`
 - 如果有多个线程等待，唤醒全部线程。

需要注意：

- `wait` 必须在加锁后调用。
- `wait` 内部会自动释放 `mutex` 并挂起，被唤醒后再自动重新加锁。
- 总是配合条件检查使用，防止虚假唤醒。
- 调用 `notify_one` 或 `notify_all` 前，一定要先更新好共享数据。

▼ 异步任务

名词	意义
<code>std::async</code>	异步地启动一个函数（可能开新线程，也可能延迟执行）
<code>std::future</code>	异步任务的结果的占位符，可以 <code>.get()</code> 来拿结果
<code>std::promise</code>	<code>std::future</code> 的一个包装，用于解决 <code>std::future</code> 不能修改
<code>std::packaged_task</code>	将函数封装，返回一个 <code>std::future</code> 对象。
<code>.get()</code>	等待异步任务执行完，并取出返回值（同步点）
<code>.wait()</code>	只等待，不取返回值

▼ 异步示例

C++

```
#include <iostream>
#include <future>
#include <chrono>

int slow_task() {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    return 42;
}

int main() {
    // 启动异步任务
    std::future<int> result = std::async(std::launch::async, slow_task);

    std::cout << "Doing other things...\n";

    // 取结果 ( 如果任务还没完成, 这里会阻塞直到拿到结果 )
    int value = result.get();

    std::cout << "Result: " << value << std::endl;
    return 0;
}
```

▼ `std::async`

`std::async` 需要设置 launch policy（启动策略）：

- `std::launch::async`：一定开新线程立刻执行。

- `std::launch::deferred`：延迟到 `.get()` 或 `.wait()` 才执行（不立即开线程）。
- 也可以不传，默认是两者择一，由系统自行决定（不推荐，可能不符合预期）。

`std::async` 基于 `std::packaged_task` 与 `std::thread` 实现：

▼ 等效伪代码

C++

```
std::packaged_task<int(int, int)> task(func);
std::future<int> result = task.get_future();
std::thread t(std::move(task), 4, 5);
```

`std::async` 本质也是对函数进行 `std::packaged_task` 封装，才能获得 `std::future` 的返回结果。

`std::packaged_task` 是一个可以存储任务（如函数、lambda 表达式、函数对象等）的对象，它是不可复制的，但可以被移动。

`std::move(task)` 将任务对象转移到另一个对象（例如，线程），而不是进行复制。

如果不使用 `std::move`，编译器会默认将对象拷贝到新线程，从而出现错误。

如果使用 `std::move`，会将任务对象的所有权转移给线程，这样线程就能安全地执行任务。

▼ std::future

`std::future` 可以理解为支持延迟赋值的容器

- `std::future` 是一个结果容器。
- 可以用 `.get()` 方法同步取值，如果异步任务没完成，会自动等待完成后取值。
- 可以使用 `.wait()` 等待线程 `std::future` 获得线程的结果。
- `.get()` 只能调用一次，调用完后 `future` 就失效了。
- 不要随便复制 `std::future`，它是不可拷贝，只能移动的。
- 如果不 `.get()` 或 `.wait()`，程序结束时 `std::future` 析构，仍会等待任务完成。
- `std::future` 的值不能被改变，但可以通过 `std::promise` 来创建一个特定的 `std::future`。

`std::promise` 可以通过 `set_value()` 设置结果，并通过 `get_future()` 获得特定结果的 `std::future` 对象。

C++

```
#include <iostream>
#include <thread>
#include <future> // std::promise std::future
using namespace std;

template<class ... Args> decltype(auto) sum(Args&&... args) {
    return (0 + ... + args);
}

template<class ... Args> void sum_thread(promise<long long> &val, Args&&... args) {
```

```

    val.set_value(sum(args...));
}

int main() {
    promise<long long> sum_value;
    thread get_sum(sum_thread<int, int, int>, ref(sum_value), 1, 10, 100);
    cout << sum_value.get_future().get() << endl;
    get_sum.join();
    return 0;
}

```

▼ 超时等待

可以用 `std::future::wait_for` 或 `wait_until` 加超时等待:

```

C++
if (result.wait_for(std::chrono::seconds(1)) == std::future_status::ready) {
    int value = result.get();
    std::cout << "Got result: " << value << std::endl;
} else {
    std::cout << "Timeout, task not ready yet." << std::endl;
}

```

- `wait_for` : 等指定时间
- `wait_until` : 等到某个时间点

▼ 线程池

实际工程中，频繁创建和销毁线程会造成额外的开销，所以会使用线程池。

线程池思想：

- 事先创建好一批线程（池子）。
- 有任务时从池子中取出线程处理。

C++ 标准库目前没有直接提供线程池，通常需要自己实现，或者用：

- `Boost::asio`
- `Intel TBB`
- `OpenMP`
- 第三方库如 `threadpool`（开源库）

▼ 轻量级线程池

```

C++
#pragma once
#include <vector>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <functional>
#include <future>
#include <iostream>

```

```

#include <type_traits>

class ThreadPool {
public:
    ThreadPool(size_t numThreads);
    ~ThreadPool();

    template<class F, class... Args>
    auto enqueue(F&& f, Args&&... args)
        -> std::future<std::invoke_result_t<F, Args...>>;

private:
    std::vector<std::thread> workers; // 线程容器
    std::queue<std::function<void()>> tasks; // 任务队列

    std::mutex queue_mutex;
    std::condition_variable condition;
    bool stop;
};

inline ThreadPool::ThreadPool(size_t numThreads) : stop(false) {
    for (size_t i = 0; i < numThreads; ++i) {
        // 直接触发隐式构造, 创建一个新的std::thread实例
        workers.emplace_back([this] {
            while (true)
            {
                std::function<void()> task;

                // 访问任务队列并从中取得一个任务, 需要加锁
                {
                    std::unique_lock<std::mutex> lock(this->queue_mutex);
                    this->condition.wait(lock, [this] {
                        return this->stop || !this->tasks.empty();
                    });

                    if (this->stop && this->tasks.empty())
                        return;

                    task = std::move(this->tasks.front());
                    this->tasks.pop();
                }

                try {
                    task(); // 线程开始执行任务
                }
                catch (const std::exception& e) {
                    std::cerr << "Exception in thread pool task: " << e.what() << "\n";
                }
                catch (...) {
                    std::cerr << "Unknown exception in thread pool task\n";
                }
            }
        });
    }
}

```

```

    }
    });
}
}

// 提交一个任务到线程池
template<class F, class... Args>
auto ThreadPool::enqueue(F&& f, Args&&... args)
-> std::future<std::invoke_result_t<F, Args...>>
{
    // std::invoke_result_t<F, Args...> 推导调用 F(Args...) 的返回类型
    using return_type = std::invoke_result_t<F, Args...>;

    // std::packaged_task<>的模板参数为函数类型，所以return_type后需要有()
    auto task = std::make_shared<std::packaged_task<return_type()>>(
        // std::bind 返回匿名函数对象，该对象将所有参数保存为成员，并通过operator()调用f(arg
        s...)
        // std::bind 返回匿名函数对象会隐式构造为std::packaged_task<return_type()>对象
        // std::forward<Args>(args)...展开为std::forward<Arg1>(arg1)、std::forward<Arg2>
        (arg2)...
        std::bind(std::forward<F>(f), std::forward<Args>(args)...)
    );

    // std::packaged_task对象与std::future对象绑定
    std::future<return_type> res = task->get_future();
    {
        std::unique_lock<std::mutex> lock(queue_mutex);

        if (stop)
            throw std::runtime_error("enqueue on stopped ThreadPool");

        // 通过一个lambda封装，隐式构造 std::function<void()>
        tasks.emplace([task]() { (*task)(); });
    }
    condition.notify_one();
    return res;
}

// 析构函数，清理线程池
inline ThreadPool::~ThreadPool() {
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        stop = true;
    }
    condition.notify_all(); // 唤醒所有线程
    for (std::thread& worker : workers)
        worker.join(); // 等待线程结束
}

```



```

int main() {
    ThreadPool pool(4); // 线程池, 4个线程

    std::vector<std::future<int>> results;

    // 提交10个任务
    for (int i = 0; i < 10; ++i) {
        results.emplace_back(
            pool.enqueue([i] {
                std::this_thread::sleep_for(std::chrono::milliseconds(100));
                std::cout << "Task " << i << " finished\n";
                return i*i;
            })
        );
    }

    // 获取结果
    for (auto && result : results)
        std::cout << "Result: " << result.get() << std::endl;

    return 0;
}

```

- 线程池中使用 `std::function<void()>` 作为任务的载体, 因为各种函数类型和函数对象的类型不同, 所以需要全部转换为 `std::function<void()>` 类型。
- `std::invoke_result_t<F, Args...>` 推导调用 `F(Args...)` 的返回类型
- `std::bind` 返回一个匿名对象, 该对象类似以下实现:

```

C++

template<class F, class... Args>
struct /*匿名*/ {
    using return_type = std::invoke_result_t<F, Args...>;
    F f_;
    std::tuple<Args...> args_;
    return_type operator()() { return std::invoke(f_, args_...); }
};

```

- `std::packaged_task` 与 `std::future` 配合实现异步任务, `std::packaged_task` 对象的 `get_future` 函数可以绑定 `std::future` 对象, `std::future` 对象的 `get()` 函数可以异步获得 `std::packaged_task` 对象的返回结果。
- 无论怎么调用, 只要 `std::packaged_task` 执行了, 就能从绑定的 `std::future` 对象处获得返回值, 所以即使 `[task]() { (*task)(); }` 没有返回值, 但是 `*task` 会被执行, 可以从绑定的 `std::future` 对象处获取返回值。
- `std::move` 并不直接释放空间, 而只是将原对象的资源指针“转交”给新对象, 并把原对象的指针置为 `nullptr`, 所以其依然存在于原本的空间, 需要使用 `pop()` 移除。

